

Guide de presentation detaille G4 — Les_Hadoop_Riders

Guide de présentation détaillé — G4 Les_Hadoop_Riders

Durée totale : 8 à 10 minutes

Règle : 2 slides par personne, dans l'ordre du groupe

Fichier PPT : PRESENTATION-G4-ORDRE.pptx (ou PRESENTATION-G4-Les_Hadoop_Riders.pptx)

Conclusion : Wren (dernière personne)

Ce guide donne le **texte complet à dire** pour chaque slide. Tu peux le lire presque tel quel, ou le reformuler avec tes mots — mais **ne saute pas les idées importantes**.

Ordre de passage

Slides	Présentateur	Contenu	Temps
1–2	Komla Petro Asinyo	Titre + plan	~1 min
3–4	Kassoum Dene	3V + HDFS vs local	~1 min 30
5–6	Joel Kazoni Tugirimana	YARN + architecture	~1 min 30
7–8	Forbes Magène	Déploiement Hub + HDFS	~1 min 30
9–10	Frank A Simo Ngounou	Job π + monitoring	~1 min 30
11–12	Wren Surprenant-Nicolson	Problèmes + conclusion	~1 min 30

Avant de commencer (tout le groupe)

1. Un seul membre change les slides (le « clicker »).
2. Chacun a **uniquement ses 2 slides** à maîtriser + les questions filet à la fin.
3. Parler **lentement**, regarder la salle, pointer l'écran quand il y a une capture.
4. Liens à connaître :
 - Docker Hub : <https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4>
 - GitHub : https://github.com/asinyopetro/Les_Hadoop_Riders_G4
 - UI HDFS : <http://localhost:9870>
 - UI YARN : <http://localhost:8088>

TEXTE COMPLET — SLIDE PAR SLIDE

KOMLA — Slides 1 et 2

Slide 1 — Titre

Temps : ~35–40 secondes

Dire exactement (ou presque) :

« Bonjour à tous. Nous sommes le groupe G4, Les_Hadoop_Riders.

Nous présentons notre projet UA1 : le déploiement et l'exploitation d'un cluster Hadoop avec Docker.

Concrètement, on a mis en place HDFS pour le stockage distribué, et YARN pour lancer des traitements.

Notre objectif est de montrer qu'on sait :

- déployer un cluster multi-nœuds,
- manipuler HDFS,
- lancer un job MapReduce sur YARN,
- et publier notre image sur Docker Hub.

À droite, vous voyez une capture de l'interface NameNode : on a bien **5 DataNodes actifs**.

Les liens Docker Hub et GitHub sont sur la slide si vous voulez les consulter. »

Puis : « Je passe au plan. »

Slide 2 — Plan

Temps : ~25–30 secondes

Dire :

« Voici le plan. Chacun a **deux slides**, dans l'ordre du groupe.

- Moi, Komla : introduction.
- Kassoum : le cas d'usage e-commerce et HDFS.
- Joel : YARN et l'architecture Docker.
- Forbes : le déploiement, Docker Hub, et les manipulations HDFS.
- Frank : le job pi et le monitoring.
- Et Wren, en dernier : les problèmes rencontrés et la **conclusion**.

On vise environ huit à dix minutes. Je laisse la parole à Kassoum. »

KASSOUM — Slides 3 et 4

Slide 3 — E-commerce et les 3V

Temps : ~45–50 secondes

Dire :

« Pour le contexte théorique, on a choisi le secteur de l'**e-commerce**.

Imaginez une chaîne de magasins qui doit gérer :

- les tickets de caisse,
- les logs du site web,
- les mouvements de stock,
- et aussi des images produits.

Ces données grossissent vite et arrivent en continu. Ça correspond aux **3V** du Big Data.

Volume : il y a beaucoup d'événements, sur plusieurs années. Une seule machine ne suffit plus.

Vélocité : les commandes et les paiements arrivent presque en temps réel. Il faut pouvoir réagir vite, par exemple en cas de fraude ou de rupture de stock.

Variété : on a plusieurs formats — CSV, JSON, logs texte, images.

Dans ce contexte, **HDFS** sert à stocker les données de façon distribuée, et **YARN** sert à lancer les traitements sur le cluster. »

Puis : « Je continue avec la différence entre HDFS et un système de fichiers classique. »

Slide 4 — HDFS vs FS classique

Temps : ~40–45 secondes

Dire :

« Voici la différence fondamentale.

À gauche : un système de fichiers **classique**, comme NTFS ou ext4.

Le fichier est stocké sur **une seule machine**. Les métadonnées sont gérées par l'OS local.

Si le disque tombe, on peut perdre les données. Il n'y a pas de réplication automatique entre plusieurs machines.

À droite : **HDFS**.

Le fichier est découpé en **blocs** — souvent 128 Mo par défaut.

Ces blocs sont **répliqués** sur plusieurs DataNodes.

Le **NameNode** garde le namespace : les noms, les dossiers, et l’emplacement des blocs.

Attention : le NameNode ne stocke pas le contenu des fichiers, seulement les métadonnées.

Donc, en résumé : le FS local est centralisé ; HDFS est distribué et plus tolérant aux pannes grâce à la réplication.

Je passe la parole à Joel. »

JOEL — Slides 5 et 6

Slide 5 — YARN (RM / NM)

Temps : ~45 secondes

Dire :

« Maintenant, YARN — Yet Another Resource Negotiator.

C’est le composant qui gère les ressources CPU et mémoire du cluster pour les jobs.

Quand on lance une application, ça se passe en trois étapes.

Un — le client : on soumet le job. Chez nous, par exemple :

```
yarn jar ... pi 4 1000 .
```

Deux — le ResourceManager, sur le master.

Il connaît les ressources du cluster, il alloue des conteneurs, et il démarre l’ApplicationMaster.

Trois — les NodeManagers, sur chaque worker.

Ils lancent les conteneurs sur leur machine, ils surveillent l’exécution, et ils remontent l’état au ResourceManager.

À retenir en une phrase :

le **ResourceManager orchestre**, le **NodeManager exécute localement**.

Sur la capture, vous voyez l’interface YARN avec les applications du cluster.

Si le professeur demande ce qu’est l’ApplicationMaster : c’est le “chef” du job ; le ResourceManager aide à le démarrer, puis l’ApplicationMaster demande les ressources pour les maps et reduces. »

Puis : « Je passe à l’architecture Docker. »

Slide 6 — Architecture Docker

Temps : ~40 secondes

Dire :

« Voici notre architecture.

En haut : le conteneur **hadoop-master**.

Il fait tourner le NameNode pour HDFS, le ResourceManager pour YARN, et aussi le SecondaryNameNode.

Les ports exposés sont :

- **9870** pour l'UI HDFS,
- **8088** pour l'UI YARN,
- **9000** pour le RPC HDFS.

En bas : **cinq workers** — worker1 à worker5.

Chaque worker a un **DataNode** pour le stockage, et un **NodeManager** pour exécuter les jobs.

Tout ça tourne sur le réseau Docker **hadoop-net**, avec la même image :

`leshadoopriders/hadoop-tp-g4:1.0` , basée sur Hadoop **3.3.6**.

Donc six conteneurs au total : un master et cinq workers.

Je laisse la parole à Forbes. »

FORBES — Slides 7 et 8

Slide 7 — Déploiement et Docker Hub

Temps : ~45 secondes

Dire :

« Pour déployer le cluster, on utilise Docker Compose.

Étape 1 : `docker compose up --build -d`

Ça construit l'image en local et démarre les six services.

Étape 2 : `docker ps`

On vérifie que le master et les cinq workers sont bien UP.

Étape 3 : `hdfs dfsadmin -report`

Le résultat attendu : **Live datanodes (5)**. Ça confirme que le cluster HDFS est opérationnel.

Ensuite, on a **publié** l'image sur Docker Hub :

`docker push leshadoopriders/hadoop-tp-g4:1.0`

Important : Docker Hub impose les **minuscules**. On ne peut pas mettre G4 en majuscules, d'où le tag `g4`.

La page publique est sur le slide — hub.docker.com/r/leshadoopridders/hadoop-tp-g4.

Le code est aussi sur GitHub.

À droite, la capture montre le terminal avec `docker ps` et `dfsadmin`. »

Puis : « Je continue avec les manipulations HDFS. »

Slide 8 — Manipulations HDFS

Temps : ~50 secondes

Dire :

« On a manipulé HDFS avec un scénario e-commerce : un dossier `/data/ventes/2026` et un fichier CSV de transactions.

1. On crée le dossier avec `mkdir -p`, puis on met les droits avec `chmod 755`.

2. On upload `transactions.csv` avec `put`, puis `chmod 644` sur le fichier.

3. On vérifie avec `fsck` et `stat` : état **HEALTHY**, **un seul bloc**, réplication initiale **3**.

Pourquoi un seul bloc ? Parce que le fichier est petit — 377 octets — alors que la taille de bloc par défaut est 128 Mo. C'est normal.

4. On lit les 5 premières lignes avec `cat | head`, puis on fusionne deux CSV avec `getmerge`.

5. On teste la suppression : avec la corbeille Trash, le fichier va dans `.Trash` au lieu d'être effacé tout de suite. Et avec `setrep -w 2`, on fait passer la réplication de 3 à 2. L'option `-w` attend que l'opération soit terminée.

Donc on a couvert : création, droits, upload, vérification, lecture, fusion, corbeille et réplication.

Je passe la parole à Frank. »

FRANK — Slides 9 et 10

Slide 9 — Job π YARN

Temps : ~45 secondes

Dire :

« Pour prouver que YARN fonctionne, on a lancé un job MapReduce officiel : le calcul de **pi** par méthode de Monte Carlo.

La commande est :

```
yarn jar suivi du jar exemples de Hadoop 3.3.6, puis pi 4 1000 .
```

Ça veut dire **4 maps** et **1000 samples** par map.

Résultat obtenu :

- Application : `application_1791039798429_0001`
- Nom : QuasiMonteCarlo
- État final : **SUCCEEDED**
- Estimation de pi environ **3,14**

Ça montre que le ResourceManager a bien alloué des ressources, et que les NodeManagers ont bien exécuté le job.

Sur la capture à droite, on voit le détail de l'application dans l'UI YARN : état FINISHED / SUCCEEDED.

On peut aussi ouvrir <http://localhost:8088>. »

Puis : « Je termine avec le monitoring. »

Slide 10 — Monitoring UI

Temps : ~40 secondes

Dire :

« Pour le monitoring, on utilise surtout les interfaces web.

À gauche : l'UI **NameNode** sur le port **9870**.

On voit l'état du DFS. Pendant nos tests, pas de missing block, et l'espace utilisé reste faible parce que c'est un labo.

À droite : l'onglet **DataNodes** — on confirme les **5 DataNodes Live**.

Côté **YARN**, port **8088** : le job pi apparaît en FINISHED / SUCCEEDED, avec les métriques mémoire et vcores.

Enfin, après le `setrep`, le NameNode met à jour la cible de réplication du fichier.

Donc on peut suivre le cluster à la fois en ligne de commande et via les UI.

Je laisse la parole à Wren pour la fin. »

WREN — Slides 11 et 12 (dernière)

Slide 11 — Problèmes et solutions

Temps : ~45 secondes

Dire :

« Avant de conclure, trois problèmes qu'on a vraiment rencontrés.

Problème 1 — pull access denied.

Compose essayait de télécharger notre image sur Docker Hub alors qu'elle n'existait pas encore.

Solution : `pull_policy: never` et build en local, puis push une fois l'image prête.

Problème 2 — majuscules dans le tag.

Docker Hub a refusé un nom avec G4 en majuscules.

Solution : tout en minuscules — `leshadoopridders/hadoop-tp-g4:1.0`.

Problème 3 — téléchargement Hadoop trop lent.

Le build restait bloqué longtemps sur archive.apache.org.

Solution : partir de l'image officielle `apache/hadoop:3.3.6`, et ajouter notre configuration XML plus le script `entrypoint.sh`.

Ces trois points montrent qu'on a débogué le déploiement, pas seulement suivi un tutoriel.

Je passe à la conclusion. »

Slide 12 — Conclusion

Temps : ~40–45 secondes

Dire :

« Pour conclure, voici ce que le groupe a livré.

Un — un cluster opérationnel : **un master et cinq DataNodes**.

Deux — HDFS maîtrisé : droits, fsck, lecture, getmerge, Trash, et setrep.

Trois — un job MapReduce pi terminé avec succès : **SUCCEEDED**, pi environ 3,14.

Quatre — une image publique sur Docker Hub : `leshadoopridders/hadoop-tp-g4:1.0`.

Cinq — un rapport, le code source, et les captures pour le rendu.

Les liens Docker Hub et GitHub sont sur la slide.

Merci de votre attention. Est-ce que vous avez des questions ? »

QUESTIONS POSSIBLES DU PROF — RÉPONSES COURTES

Question	Qui peut répondre	Réponse courte
Combien de DataNodes ?	Tout le monde	5
Quelle version de Hadoop ?	Joel / Frank	3.3.6
Nom de l'image Hub ?	Forbes / Wren	leshadoopridders/hadoop-tp-g4:1.0
Où est le CSV sur HDFS ?	Forbes	/data/ventes/2026/transactions.csv
Différence HDFS / local ?	Kassoum	Blocs + réplication + NameNode
Qui alloue les ressources ?	Joel	Le ResourceManager
C'est quoi un bloc ?	Kassoum / Forbes	Morceau du fichier (souvent 128 Mo)
Pourquoi 1 seul bloc sur le CSV ?	Forbes	Fichier trop petit (377 o)
À quoi sert Trash ?	Forbes	Évite la suppression définitive immédiate
Pourquoi pi ?	Frank	Exemple officiel Hadoop, simple, prouve YARN
Pourquoi g4 en minuscules ?	Forbes / Wren	Docker Hub refuse les majuscules
Qu'est-ce que l'ApplicationMaster ?	Joel	Chef du job, démarré via le ResourceManager

Si tu ne sais pas : « On a détaillé ça dans le rapport, section ... » — mieux que d'inventer.

CONSEILS PRATIQUES

1. **Ne lis pas le slide mot à mot** si le texte est déjà affiché — parle comme à l'oral, en t'aidant du script.
2. **Pointe** les captures quand tu dis « à droite » / « à gauche ».
3. Si tu dépasses le temps : coupe un détail, **jamais** les idées clés (5 DataNodes, SUCCEEDED, Hub).
4. Entre deux personnes : une phrase courte de passage (« Je laisse la parole à ... »).

5. Pendant les questions : laisse parler la personne concernée ; les autres peuvent compléter après.

CHECKLIST JOUR J

- ☐ PPT ouvert et testé en mode diaporama
- ☐ Chacun a relu **ses 2 slides** à voix haute une fois
- ☐ Komla prêt à démarrer / Wren prêt à conclure
- ☐ Rapport PDF accessible si le prof demande une preuve
- ☐ Téléphones en silencieux
- ☐ Parler lentement, regarder la salle